



Total Marks: 3

Obtained Marks: _____

Parallel and Distributed Computing Assignment # 04

Last date of Submission: 05th June, 2026

Submitted To: Asma Niaz Awan _____

Student Name: _____

Reg. Number: _____

COMPUTER SCIENCE DEPARTMENT

Instructions: Copied or shown assignments will be marked zero. Late submissions are not entertained in any case. Assignments should be submitted on due date and time

Assignment Task:

- Q1.** Differentiate between Uniform Memory Access (UMA), Non-Uniform Memory Access (NUMA), Symmetric Multiprocessing (SMP) and Massively Parallel Processing (MPP) architectures with respect to memory access time, scalability, programming implications, and one suitable application/use case for each architecture.
- Q2.** Explain why Apache Spark, Apache Flink is usually more suitable than Hadoop MapReduce for iterative and streaming data processing. Also mention one consistency, fault-tolerance, or security concern in distributed data processing.
- Q3.** A client needs the current time from a server using the ZeroMQ request-reply pattern. Write Python code or clear pseudocode for both components:
- Server: create a ZeroMQ context, create a REP socket, bind it to tcp://HOST:PORT, wait for requests, reply with time.asctime() when the request is TIME, and send an error message for invalid requests.
 - Client: create a ZeroMQ context, create a REQ socket, connect to the same address, send the literal string TIME, receive the reply, and print the timestamp.
- Q4.** A distributed program is executed using two MPI processes. Process with rank 0 prepares a Python dictionary and sends it to the process with rank 1. Consider the following two mpi4py programs.

Program 1: Synchronous Message Transfer

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    message = {"student_id": 101, "score": 89.5}
    comm.send(message, dest=1, tag=20)
elif rank == 1:
    received_message = comm.recv(source=0, tag=20)
```

Program 2: Asynchronous Message Transfer

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    message = {"student_id": 101, "score": 89.5}
    request = comm.isend(message, dest=1, tag=20)
    request.wait()
```

COMPUTER SCIENCE DEPARTMENT

```
elif rank == 1:  
    request = comm.irecv(source=0, tag=20)  
    received_message = request.wait()
```

- Briefly explain the working of both programs. Identify which process sends the dictionary and which process receives it.
- Differentiate between synchronous/blocking communication and asynchronous/nonblocking communication in MPI with reference to Program 1 and Program 2